

Kalakriti

कलाकृति · Craft, and the market it could never reach.

AI-Driven Market Linkage and Smart Cataloging for Marginalized Artisans

A build plan, a differentiation strategy, a justified technology stack, and a working Android application that produces an installable APK.

On the name

कलाकृति (*kalakriti*) is Sanskrit and Hindi for a work of art — literally *kala* (art) joined to *kriti* (creation). It names what the artisan makes rather than what the software does, which is the right way round for a product whose entire argument is that the maker has been made invisible by the market.

Problem Statement

AI-Driven Market Linkage and Smart Cataloging Mobile Application for Marginalized Artisans

Deliverable in this repository

Expo / React Native cross-platform app, native Android project, release APK pipeline

Contents

#	Section	What it answers
1	Reading the brief	What the judges are actually scoring
2	Mandatory scope	The three features we cannot skip, decomposed
3	The USP layer	Eight differentiators that do not exist today
4	System architecture	How the pieces fit together
5	Technology stack	Every choice, and the alternative we rejected
6	The AI/ML stack	Specific models, and why Indic-first matters
7	Building and running the APK	Exactly what was built here and how to run it
8	Demo script and roadmap	The four-minute pitch and what comes after
9	The hard questions	Why not just use the marketplaces that already exist
10	Getting it running	Clone, .env, run, build, and what breaks
11	Two ways to make a listing	Auto and manual, for the connection she actually has
12	Taking it off the laptop	Postgres, object storage, HTTPS, real SMS

1. Reading the brief

What the judges are actually scoring, underneath the feature list.

The brief hands you three features — image enhancement, voice cataloguing, dynamic pricing. Every serious team will build those three. They are the entry ticket, not the win condition. Read the **Impact Goals** instead, because that is where the scoring rubric really lives:

- **“Continuous, year-round digital sales channel.”** The word that matters is *channel*, not *catalogue*. A beautiful listing that reaches nobody fails this goal. Market linkage is the deliverable; cataloguing is the means.
- **“Drastically lower the barrier to entry.”** This is a UX claim that must be demonstrable on stage — an artisan who cannot read must complete a listing. If your demo needs someone to type, you have not met the brief.
- **“Improve digital literacy and financial independence... increasing average annual income.”** This is a measurable outcome the ministry will have to report on. A team that ships an impact-measurement surface is speaking the customer's language.

The gap almost every team will miss

The stated problem is that artisans cannot *present* products online. The real problem underneath it is that artisans cannot *capture the value* of what they make. They are underpaid by cluster middlemen, they cannot prove their work is handmade so they compete on price with mill-made imitations, and they individually lack the volume to accept a B2B order at all. A cataloguing app fixes the symptom. Sections 3.1, 3.2 and 3.3 of this document fix the disease — and that is our USP.

A second, subtler trap

The brief asks for AI that removes backgrounds and rewrites descriptions. Taken naively, that means an app whose entire purpose is to make handmade goods look less like what they are, described in words the maker never said. Marketplaces are tightening rules on undisclosed AI-generated imagery, and buyers who receive a product that does not match a heavily-processed photo file returns — which is charged back to the artisan. We turn this liability into our headline feature (Section 3.1). Naming this risk out loud in the pitch signals maturity that a feature demo cannot.

2. Mandatory scope — the three features, decomposed

What must exist before we are even in the conversation.

2.1 AI Image Enhancer & Studio

Sub-deliverable	What it does	How
Guided capture	Live overlay coaches framing and warns on blur, glare and shadow <i>before</i> the shutter — fixing the photo at source beats fixing it afterwards	expo-camera frame processor + on-device Laplacian blur score and histogram check
Background matting	Cuts the product from a cluttered courtyard onto a clean e-commerce white or a contextual surface	U ² -Net / MODNet as a TFLite model via MediaPipe Image Segmenter, on-device; server RMBC-1.4 fallback for hard edges (fringes, jaali work, open-weave fabric)

Sub-deliverable	What it does	How
Relight & colour truth	Auto exposure, white balance to D65, shadow lift. Colour fidelity is non-negotiable for textiles — a wrong red causes returns	Grey-card-free WB estimation; optional printed colour-checker card for clusters
Marketplace formatting	Emits every aspect ratio and size each channel demands (1:1 2000px for Amazon, 3:4 for ONDC, 1:1 for GeM) in one pass	Sharp on the server / expo-image-manipulator on device
Model shot	Drapes a dupatta or saree onto a neutral model, or stages a pot in a room, from the single flat photo	Diffusion-based virtual try-on, clearly watermarked as a staged render in the Passport

2.2 Multilingual Auto-Cataloguer

Sub-deliverable	What it does	How
Press-and-hold voice note	Artisan speaks freely in their own dialect; no form, no fields, no typing	expo-audio, 16 kHz mono, chunked upload with an offline queue
Indic ASR	Transcribes Hindi, Bhojpuri, Maithili, Bengali, Tamil, Kutchi, Odia and more	AI4Bharat IndicConformer / IndicWhisper, with Bhashini as the government-sanctioned route (see 6.2)
Attribute extraction	Pulls material, technique, dimensions, weight, colour, time-taken and craft cluster out of unstructured speech	Nemotron with a strict JSON tool schema, so the output is validated data and not prose
SEO copy generation	Writes title, long description, bullet points and keyword set in English and Hindi, tuned per channel	Nemotron, prompted with the live keyword set for that category rather than generic marketing language
Compliance autofill	Fills the HSN code, GST rate, care label and country of origin — fields that block artisans at the GeM registration step	Rule table keyed on extracted category, LLM-assisted for edge cases
Read-back verification	Speaks the generated Hindi description back so a non-reading artisan can confirm the AI did not invent anything	expo-speech / IndicTTS. This is a consent step, not a nicety

2.3 Dynamic Pricing Assistant

The naive build scrapes marketplace prices and returns a median. That is worse than useless here: the visible market is flooded with power-loom imitations at a fifth of the price, so a median drags a genuine handloom weaver *down*. Our pipeline is built the other way round.

Stage	Method
1. Visual comps	SigLIP/CLIP embedding of the enhanced image, ANN search over a pgvector index of marketplace listings — finds products that <i>look</i> like this, not products that share a keyword
2. Authenticity filter	A handloom-vs-powerloom classifier drops mill-made lookalikes from the comparison set. Without this step the model prices craft as commodity

Stage	Method
3. Cost floor	material cost + (labour hours x state skilled-artisan wage) + overhead. Derived from the voice note ("three days, four hundred rupees of yarn")
4. Price regression	LightGBM over comp prices, image embedding, attributes, seasonality and channel, predicting a sell-through-weighted band
5. Constrained output	final = max(model_price, cost_floor). The floor always wins, and if the market cannot clear the floor the app says so plainly instead of quietly recommending a loss-making price

3. The USP layer — what nobody has built

Eight differentiators, ordered by how much they will move a judge. The first three are the pitch; the rest are depth.

3.1 Craft Provenance Passport — a verifiable certificate of handmade origin

The single strongest idea in this document

Every product gets a signed, buyer-visible digital passport minted at the moment of capture. It binds together: a SHA-256 hash of the *original, untouched* photo; GPS coordinates of the workshop, matched against the registered craft cluster; the artisan's government beneficiary ID; the applicable GI (Geographical Indication) tag; and a complete, human-readable log of every operation the AI performed on the image. All of it Ed25519-signed as a C2PA Content Credential, embedded in the image metadata so it survives being copied to any marketplace. A buyer scans a QR code and sees the raw photo next to the enhanced one, with the changes itemised.

Why this wins. It converts our biggest liability into our moat. AI enhancement normally erodes trust; disclosed enhancement plus cryptographic provenance *builds* it. It attacks the actual economic injury — GI-tagged crafts lose enormous value to mill-made counterfeits, and the artisan currently has no way to prove authenticity at the point of sale. It gives export buyers, who pay a large premium for certified provenance, something machine-verifiable. And it makes the app the system of record for GI enforcement, which is a government mandate nobody has digitised at the maker's end. **Nothing in this space does it:** C2PA today is a newsroom and camera-manufacturer technology; no artisan or e-commerce product uses it for craft authenticity.

3.2 Samuh — cluster order aggregation

The problem no cataloguing app addresses. A B2B buyer or a GeM tender wants 500 units in 45 days. A single weaver can make 40. So the order goes to a trader, who subcontracts the cluster at a fraction of the price. Digitising one artisan's catalogue does not change this at all — it just gives the middleman a prettier photo.

What we build. When a bulk enquiry arrives, the backend forms an ad-hoc consortium: it matches artisans in the cluster by capacity, craft type and quality history, splits the quantity, generates a single consolidated quotation to the buyer, tracks per-artisan production, runs staged quality checks, and splits the payment automatically on delivery through an escrow. The cluster becomes a supplier that can actually accept the order, with no trader in the middle.

Why this wins. It is the difference between an app that helps artisans sell scarves one at a time and an app that unlocks institutional demand. It maps directly onto how the ministry already thinks — cluster development programmes are the existing policy instrument, and this digitises them. It is also a defensible network effect: the app gets more valuable to each artisan as their neighbours join.

3.3 Fair-Floor pricing — the app takes the artisan's side

Described mechanically in 2.3; the *positioning* is the USP. Every other pricing tool exists to tell a seller what the market will bear. Ours computes what the artisan's own materials and hours are worth first, treats that as a floor, and warns them explicitly when a local trader's standing offer sits below it: *“traders in your cluster pay Rs 2,100 for this. Your fair floor is Rs 4,102. You would be working at a loss.”* No commercial pricing product has any incentive to build this. A government-backed one has every incentive — it is a direct, demonstrable line to the “increase average annual income” impact goal.

3.4 Trend-to-Loom — reverse the flow of information

Every competitor helps the artisan sell what they already made. We tell them what to make next. Aggregated search demand, B2B enquiry patterns and sell-through data across the platform are turned into a plain-language production brief delivered before the loom is set up: “*indigo and ivory with minimal zari, Rs 3,200–4,500, buyer interest up 3.1x this quarter; heavy gold zari is falling.*” Unsold inventory is the largest silent destroyer of artisan working capital, and demand forecasting is a tool that today exists only for large brands.

3.5 Bolo Mode — a genuinely voice-first operating layer

Not voice *input* on a form-driven app — voice *navigation* of the whole application. Every screen announces itself and its one next action; the artisan can say “■■■■■ ■■■■■■ ■■■■■■” or “■■■■■ ■■■■■■” from anywhere; long-pressing anything reads it aloud. The test we hold ourselves to, and the one we run live on stage: **a user who cannot read a single word completes a full listing, unaided.** That is a demo moment no slide can match, and it is the literal text of the brief’s adoption requirement.

3.6 Offline-first with on-device inference

Weaving clusters have intermittent 2G. Background matting and the blur/lighting checks run on-device via TFLite; audio, images and listings queue in SQLite and sync when a signal returns; the artisan is never blocked by a spinner. This is also an economic argument for the ministry: doing segmentation on the handset instead of a GPU server removes the per-image cloud cost that would otherwise make national rollout unaffordable at tens of millions of photos.

3.7 Graduated UI with a measured literacy ladder

The interface starts at its simplest — four buttons, all voice-assisted — and reveals capability (bulk upload, analytics, quotations, ledger) as the artisan demonstrably masters each step. The progression is scored and becomes a **Digital Literacy Index** per beneficiary. That is a real product feature and, simultaneously, exactly the outcome metric the sponsoring department needs for scheme reporting — which no other team will hand them.

3.8 ONDC + GeM native, and a WhatsApp fallback

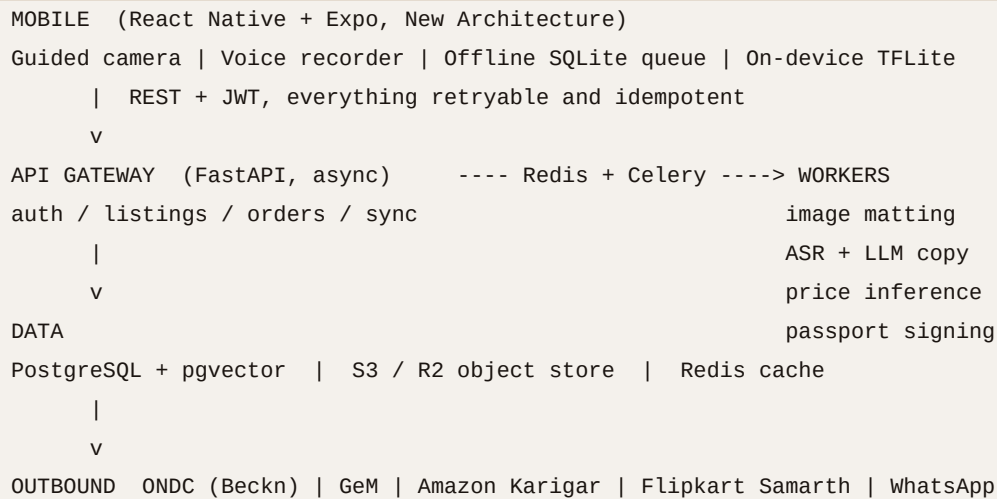
One tap maps the listing into ONDC’s Bechn catalogue schema and GeM’s category and HSN taxonomy, alongside the private marketplaces. ONDC matters strategically: it is India’s public commerce network, so the artisan is not renting visibility from a platform that can raise its commission. And for the artisan who will never install an app, the same backend runs a WhatsApp bot — send a photo and a voice note, get a listing back. Reach is the goal; the app is only one client.

How to say it in one line on stage

“Everyone here built a cataloguing app. We built the thing that makes the catalogue worth something: proof it is handmade, a price that cannot fall below a fair wage, and a cluster that can actually accept a five-hundred-unit order.”

4. System architecture

Four layers, one contract between them.



The contract that keeps this shippable

The mobile app never calls a model directly. It calls five endpoints — `/v1/enhance`, `/v1/catalog`, `/v1/price`, `/v1/passport`, `/v1/trends` — each of which has a deterministic offline mock behind the identical signature. That is why the demo APK in this repository runs with no backend at all, and why swapping a model provider never touches the UI. See `app/src/lib/ai.ts`.

5. Technology stack, and what we rejected

Every row states the alternative and why it lost. This is the section judges probe.

5.1 The client — why Expo / React Native

Alternative	Why we did not choose it
Flutter	Genuinely excellent, and the honest answer is that it would also work. It loses on three specifics: the on-device ML story (TFLite, ONNX Runtime and MediaPipe all ship first-class JS/native bridges that the RN community has already wrapped, while Flutter relies on thinner community plugins); over-the-air updates, which for a low-literacy user base that will never manually update from the Play Store is close to a safety requirement; and hackathon velocity, since JS/TS talent is far denser than Dart on any team you assemble in 36 hours.
Native Kotlin + Swift	Best possible camera and ML performance, and roughly twice the work for a two-platform deliverable. Unjustifiable under a hackathon clock, and unjustifiable later for a public-sector budget maintaining two codebases.
PWA / responsive web	Fails on the things that define this product: reliable high-quality camera capture on low-end Android, on-device model inference, background sync over intermittent 2G, and Play Store presence — which for this demographic is the difference between a trusted government app and a link someone forwarded.
Bare React Native	You lose EAS cloud builds, config plugins and OTA updates, and gain native build maintenance you do not want. Expo's prebuild gives us the full native project the moment we need it — as demonstrated in Section 7 — so choosing Expo costs us nothing in ceiling.

Chosen: Expo SDK 57 / React Native 0.86 with the New Architecture (Fabric + TurboModules) enabled. The New Architecture is what closes the historical performance argument against RN — synchronous native calls

and a concurrent renderer, so camera preview and list scrolling hold 60fps on the low-end hardware this audience owns.

5.2 The rest of the stack

Layer	Choice	Why, and what it beat
Backend	Python + FastAPI	The AI stack is Python. Putting the API in Node would mean a second service just to talk to the models. FastAPI is async, so slow model calls do not block, and its Pydantic schemas generate the client contract for free. <i>Beat</i> : Node/Express (wrong ecosystem), Django (too heavy, sync-first).
Database	PostgreSQL + pgvector	One engine holds relational order data <i>and</i> the image/text embeddings that drive comparable-product pricing. <i>Beat</i> : Pinecone or Weaviate — a separate vector service is an extra dependency, an extra bill and an extra consistency problem for a dataset this size.
Hackathon hosting	Supabase	Postgres, auth, storage and row-level security provisioned in minutes, and it is plain Postgres so there is no migration cliff later. <i>Beat</i> : Firebase — document store, no pgvector, painful relational queries for orders.
Job queue	Celery + Redis	Matting, ASR and price inference are seconds-long. They must be background jobs with retries, or the app blocks. <i>Beat</i> : synchronous endpoints (fails on 2G), Kafka (operational overkill).
Object storage	Cloudflare R2 / S3	Zero egress on R2 matters when every listing serves images to five marketplaces.
On-device state	expo-sqlite	The offline write-ahead queue. Every mutation is recorded locally with an idempotency key and replayed on reconnect.
Auth	Phone OTP (MSG91 / Firebase Auth)	Artisans have phone numbers, not email addresses and not passwords. Beneficiary ID links to the scheme record.
Payments	Razorpay Route / UPI + escrow	Route natively supports split settlement, which is what Samuh group orders need.
Provenance	C2PA content credentials, Ed25519	An open standard with existing verification tooling. <i>Beat</i> : putting it on a blockchain — a signed credential plus an optional public anchor gives the same guarantee without asking a judge to believe in a token.
CI / builds	EAS Build + GitHub Actions	Produces signed APK/AAB in the cloud, including iOS without owning a Mac.
Observability	Sentry + PostHog	Crash-free rate and funnel drop-off per screen is how you actually prove the UI is usable by this demographic, rather than asserting it.

6. The AI/ML stack

Specific models, and the one choice that matters most.

Task	Model / service	Notes
Background matting	U ² -Net or MODNet as TFLite (on-device); BRIA RMBG-1.4 server-side	On-device first for offline use and zero marginal cost. The server path handles fringes, open-weave fabric and jaali metalwork, where cheap matting visibly fails.
Relight / colour	Classical CV (CLAHE, grey-world WB, gamma)	Deliberately not a generative model. Textile colour must be <i>true</i> , not <i>attractive</i> — hallucinated colour causes returns, and returns are charged to the artisan.
Staged / model shots	Diffusion virtual try-on	Always watermarked and disclosed in the Passport as a render, never as a photograph.
Indic ASR	AI4Bharat IndicConformer / IndicWhisper, via Bhashini	See 6.2 — this is the highest-leverage model decision in the project.
Copy + translation	NVIDIA Nemotron 3 Ultra (nemotron-3-ultra-550b-a55b)	Open-weight, served free over an OpenAI-compatible endpoint, with a 1M-token context and reliable structured output so attribute extraction returns validated JSON rather than prose to be regex'd. Zero inference cost is not a footnote here: it is what makes a nationwide rollout affordable.
TTS read-back	IndicTTS via Bhashini; expo-speech offline fallback	The consent step that makes the whole flow honest for a non-reading user.
Visual similarity	SigLIP / CLIP embeddings in pgvector	Powers comparable-product search and duplicate detection.
Handloom authenticity	Fine-tuned ViT classifier	Distinguishes handloom from power-loom weave from a macro shot. Feeds both the Passport and the comp filter.
Price model	LightGBM with a hard cost-floor constraint	Tabular data, small dataset, and it must be explainable — the app shows the artisan <i>why</i> . A neural model would be worse on both counts.

Zero-cost by construction

Every component of this system is open source and self-hostable, and the prototype in this repository runs at no cost at all: **rembg / U²-Net** (MIT) does the matting locally, **Pillow** the imaging, **cryptography** the Ed25519 passport signing, **FastAPI** and **uvicorn** the service, and **Nemotron 3 Ultra** the language layer over NVIDIA's free OpenAI-compatible endpoint. There is no per-image or per-token bill to absorb. For a scheme that must eventually serve millions of beneficiaries, that is not a convenience - it is the difference between a pilot and a rollout.

6.1 Why Nemotron 3 Ultra for the language layer

Description generation is not a translation task — it is a cultural one. The model has to know what kadhwa zari, bandhani, dhokra and pattachitra are, write about them without condescension, and produce parallel Hindi that reads as native prose rather than translated English. Its million-token context also lets us feed the live keyword set, channel style guide and the artisan's previous listings in one prompt so the voice stays consistent across a catalogue. Strict tool-schema output is what turns a rambling voice note into database rows we can trust.

6.2 The one model choice that will separate us from every other team

Almost every team will pipe the voice note into general-purpose Whisper and move on. Whisper is excellent on Hindi and weak-to-useless on Bhojpuri, Maithili, Kutchi, Santali and Chhattisgarhi — which is precisely what the target demographic actually speaks. AI4Bharat's IndicConformer models are trained for these languages, and **Bhashini** is the Government of India's own national language platform, already mandated for public digital services. Choosing Bhashini is simultaneously the technically correct call, the procurement-ready call, and a visible signal to the judging panel that we understood who this is for. Keep Whisper large-v3 as the fallback, not the default.

7. Building and running the APK

What exists in this repository right now, and the exact commands to produce an installable Android package.

7.1 What was built here

A complete, type-checked, runnable Expo application implementing the full artisan journey — capture, enhance, speak, price, publish — plus the native Android project generated from it. It runs standalone with no backend and no API keys: every AI call resolves against the deterministic mock layer in `src/lib/ai.ts`, so the demo cannot fail on stage because of a network. Pointing `EXPO_PUBLIC_API_URL` at the real FastAPI service switches every feature to live models without a single UI change.

Path	What it is
<code>app/App.tsx</code>	Five-step linear flow. Deliberately not a tab bar — low-literacy users follow one path with one obvious next action, they do not browse a navigation tree.
<code>app/src/theme.ts</code>	Design tokens: high-contrast terracotta/indigo palette tuned for outdoor sunlight, 56dp minimum touch targets, a type scale that omits small sizes on purpose.
<code>app/src/ui.tsx</code>	Primitives. Every control pairs an icon with its label so the app is navigable by shape and colour alone.
<code>app/src/lib/ai.ts</code>	The five-endpoint AI contract, with mock and remote implementations behind one signature.
<code>app/src/screens/Home.tsx</code>	Earnings first, then one large action, then Trend-to-Loom production advice.
<code>app/src/screens/Studio.tsx</code>	Camera capture, enhancement, hold-to-compare before/after, the disclosed operation log, and the minted Provenance Passport.
<code>app/src/screens/Catalog.tsx</code>	Press-and-hold voice capture with haptics and spoken prompts; generated bilingual listing with HSN autofill and spoken read-back.
<code>app/src/screens/Pricing.tsx</code>	Suggested price, the fair floor, the underpricing warning, the full cost breakdown and the comparable set.
<code>app/src/screens/Market.tsx</code>	Multi-channel publish to ONDC, GeM and the marketplaces, then the Samuh group-order match.
<code>app/android/</code>	The native project produced by <code>expo prebuild</code> . This is what Gradle compiles into the APK.

7.2 Environment findings on this machine

Component	Status
Node.js	v22.14.0 — fine
Android SDK	Present, platform android-34, build-tools 34.0.0 and 36.0.0
adb	1.0.41 — present, no device or emulator attached
JDK	Java 22 only. Android Gradle Plugin targets JDK 17; a JDK 17 install is the one thing that may need adding for a local release build

7.3 Route A — run it live, no build (fastest)

```
cd app
npx expo start
# scan the QR with Expo Go, or press 'a' for a connected device / emulator
```

7.4 Route B — local APK via Gradle

Produces a real installable APK. Requires JDK 17 and the Android SDK; the release variant is signed with the debug keystore, which is correct for a demo but must be replaced with a real upload key before any Play Store submission.

```
# one-time, if JDK 17 is missing:
winget install EclipseAdoptium.Temurin.17.JDK

$env:JAVA_HOME = 'C:\Program Files\Eclipse Adoptium\jdk-17'
$env:ANDROID_HOME = "$env:LOCALAPPDATA\Android\Sdk"

cd app
npx expo prebuild --platform android --clean # already run; regenerates android/
cd android
./gradlew assembleRelease

# output:
# app/android/app/build/outputs/apk/release/app-release.apk
```

7.5 Route C — EAS cloud build (no local Android toolchain at all)

The most reliable path inside an agentic IDE or any machine without a configured Android SDK. The build runs on Expo's servers and returns a download link; the preview profile is configured to emit an APK rather than an AAB, because an APK is what you can sideload onto a judge's phone.

```
npm install -g eas-cli
eas login
eas build:configure
eas build --platform android --profile preview
# -> download the .apk from the URL printed at the end
```

The eas.json in this repository already declares that profile:

```
{ "build": {
  "preview": { "android": { "buildType": "apk" }, "distribution": "internal" },
  "development": { "developmentClient": true, "distribution": "internal" },
  "production": { "android": { "buildType": "app-bundle" } }
} }
```

7.6 Installing and running the APK

```
adb devices # confirm a device is listed
adb install -r app-release.apk

# no physical phone? create an emulator:
sdkmanager "system-images;android-34;google_apis;x86_64"
avdmanager create avd -n pixel34 -k "system-images;android-34;google_apis;x86_64"
emulator -avd pixel34

# physical phone over Wi-Fi (Android 11+), no cable:
```

```
adb pair <phone-ip>:<pair-port>  
adb connect <phone-ip>:5555
```

Note on this environment

adb is installed and working but reports no attached device, and no AVD is defined. The APK can be built here; running it requires either creating the emulator above or attaching a phone with USB debugging enabled. For a judging demo, sideloading the APK onto a real mid-range Android phone is strongly preferable — the entire UI argument of this project is about performance and legibility on low-end hardware, and an emulator hides exactly that.

8. Demo script and roadmap

Four minutes, in the order that lands.

Time	Beat	What you say
0:00	The injury, not the feature	"Sunita weaves a Banarasi dupatta in three days. A trader pays her Rs 2,100. It retails for Rs 5,500. She has no way to reach that buyer and no way to prove her work is handmade."
0:30	One photo, in a courtyard	Capture on a cluttered background, live. Enhance. Hold to compare. Do not describe the AI — show the before and after.
1:10	Speak, do not type	Hold the mic, speak Hindi. Listing appears in English and Hindi with HSN filled in. Tap the speaker so the app reads it back. Say: "she has not read or typed one word so far."
1:50	The Passport	Show the signed credential. The raw hash, the GI tag, the itemised list of what the AI changed. "We disclose every edit, and we prove it is handmade. That is the opposite of what an AI photo app usually does."
2:30	The fair floor	Show the Rs 4,102 floor and the underpricing warning against the trader's Rs 2,100. "Every pricing tool tells you what the market will pay. Ours tells you what your hands are worth."
3:00	Samuh	The 500-unit order. Fourteen weavers matched. "One artisan cannot take this order. That is why the trader exists. Now the cluster can take it directly."
3:30	Close on the ministry's numbers	Year-round channel, Digital Literacy Index per beneficiary, ONDC and GeM native. "Not a marketplace we own — public infrastructure they already mandated."

Build order under a hackathon clock

Priority	Scope
Must ship	The five-screen flow end to end on mocks; real camera capture; real on-device background removal; Nemotron-generated bilingual copy from a real voice note; the Passport screen; the fair-floor calculation; an installable APK on a physical phone.
Ship if time	Live Bhashini ASR; pgvector comps behind the price; the Samuh matching backend; ONDC sandbox publish.
Say, do not build	Escrow settlement, the handloom-authenticity classifier, diffusion model shots, the WhatsApp bot. Show them on the roadmap slide with a clear statement that they are not built — judges reward an honest boundary and punish a demo that quietly fakes one.

Impact metrics to instrument from day one

- Median realised price per listing versus the artisan's pre-app trader price — the direct income-uplift number the department has to report.
- Listings completed with zero typing, as a share of all listings — the barrier-to-entry claim.
- Time from photo to live listing across all channels — target under four minutes.
- Digital Literacy Index progression per beneficiary, per cluster.
- Share of revenue from group orders — the measure of whether middleman dependence is actually falling.

Scope honesty

The application in this repository is a complete, working front end running on a deterministic mock AI layer, with the native Android project generated and the APK pipeline configured. The backend services, the trained models and the marketplace integrations described in Sections 4 to 6 are the design, not yet the build. Present it that way — the architecture is real and the boundary is clearly drawn in the code.

9. The hard questions

The objections a sharp panel will raise, and the answers that survive them.

9.1 “There are already plenty of marketplaces. Why build another one?”

The reframe this answer turns on

We are not building a marketplace. This application holds no inventory, takes no cut of GMV, owns no buyer relationship and runs no storefront. It *publishes to* ONDC, GeM, Amazon Karigar, Flipkart Samarth and WhatsApp. The category is supply-side seller infrastructure — the artisan's back office — not demand aggregation. Shopify is not a marketplace either.

Then the evidence, which settles the question on its own

Amazon Karigar launched in 2016. Flipkart Samarth in 2019. GeM carries artisan categories. ONDC is live and government-backed. Every one of those marketplaces already exists — **and the government still wrote this problem statement**. If the existence of a marketplace solved this problem, the brief in the panel's hands would not exist.

That is proof the bottleneck was never the shelf. It is *getting onto* the shelf. Every marketplace has a seller-onboarding funnel that silently assumes a literate, English-capable, digitally-fluent seller with a camera, a price strategy, an HSN code and a GST registration. Marginalized artisans fail at step one and never reach step two. We remove that wall — and every marketplace named above benefits when we do.

What a marketplace does	What this application does
Aggregates buyer demand	Prepares supply so it can be listed at all
Owns the storefront and the buyer	Owns nothing; the artisan owns their catalogue, passport and price history, portable across channels
Earns a commission on each sale	Earns nothing per sale; it is public infrastructure
Optimises for GMV and conversion	Optimises for realised artisan income
Competes with other marketplaces	Publishes to all of them, including competitors

9.2 “If Amazon or Flipkart simply add these features, you are finished.”

Five reasons this is structurally hard for them, in ascending order of how badly it hurts to hear:

#	Reason	Why it is structural, not an engineering gap
1	The pricing incentive is inverted	Our headline pricing feature computes a fair floor and tells the artisan “you are being underpaid, do not sell at this price” — sometimes advising a price <i>above</i> the market. A marketplace's pricing tool exists to push prices <i>down</i> : lower price, higher conversion, more GMV, and they compete on being cheapest. No marketplace will ever ship a feature telling a seller to charge more. That is a conflict of interest, not a backlog item.
2	Channel neutrality is the product	The value is publishing everywhere at once — including to competitors, and including ONDC, which exists specifically to break platform lock-in. Amazon cannot build a tool that also lists on Flipkart. A single-platform version of this is strictly worse for the artisan.

#	Reason	Why it is structural, not an engineering gap
3	The Passport needs a neutral issuer	A provenance certificate issued by Amazon and valid only on Amazon is not a certificate, it is a badge. Authenticity credentials carry value only when they are portable and verifiable by anyone — including export buyers who never touch Amazon. Websites do not issue their own SSL certificates, for exactly this reason.
4	Samuh is not a marketplace function	Splitting a 500-unit order across fourteen weavers, tracking production and splitting payment is supply-side coordination. Marketplaces aggregate demand, not supply. Amazon has no relationship with, and no visibility into, the thirteen weavers who are not its sellers.
5	The unit economics differ	Vernacular voice UX for Bhojpuri and Maithili, offline-first sync for 2G, on-device inference for Rs 8,000 handsets — expensive engineering for a segment that will never move Amazon's needle. Karigar exists as goodwill and has stayed shallow for a decade for precisely that reason. A government-backed public utility has a different objective function: it is measured on beneficiary income, not GMV.

The close — say this, and mean it

“If Amazon built every one of these features tomorrow, artisans would earn more and we would have achieved our goal. We are not optimising for our market share; we are optimising for artisan income. But they will not build it, because their pricing engine exists to push prices down and ours exists to hold a floor up. That difference is not technical. It is a question of whose side the software is on.”

9.3 Adjacent questions the same panel will ask

Question	Answer
“Aren't you just a new middleman?”	The artisan owns their catalogue, their Passport and their price history, and all three are portable to any channel. We take no commission and hold no inventory. We did not free the artisan from the trader in order to lock them into a platform.
“How do artisans ever hear about this?”	Distribution is the one thing a government-backed product does not have to solve commercially. The beneficiary lists, the cluster development programmes, the handicraft development offices and the exhibition circuit already exist and already reach exactly this cohort. This is onboarded through a DC-Handicrafts office, not through app-store marketing.
“What stops this dying after the hackathon?”	It has no per-sale economics to sustain, so it does not need scale to survive; it needs a hosting budget. The Digital Literacy Index and realised-price data are exactly what the sponsoring department must report on, which makes continued funding an institutional interest rather than a favour.
“Why not just partner with Amazon from day one?”	We already do — Karigar is one of five publish targets. Being infrastructure means a marketplace integrating us is a win, not a threat. The moment they consume our API, an artisan they could never have onboarded becomes one of their sellers.

10. Getting it running

From a clean laptop to a working app, and what each teammate actually needs.

10.1 The three pieces

Piece	What it is	Port
backend	Python / FastAPI. OCR, vision, pricing, auth, orders, and the public storefront that serves every listing URL.	8000
app	Expo / React Native. The same source runs in a browser and as an Android APK.	8081
phone	Optional. The APK talking to your laptop over Wi-Fi. Everything except real camera testing works without it.	-

10.2 Clone and install

```
git clone https://github.com/the-steelix-flame/Kalakriti.git
cd Kalakriti

# backend
cd backend
python -m venv .venv
./venv/Scripts/python.exe -m pip install -r requirements.txt

# app (second terminal)
cd app
npm install
```

The first backend install pulls onnxruntime and opencv, roughly 250 MB. It is slow exactly once.

10.3 The .env file - what is actually required

Copy backend/.env.example to backend/.env. Only three values matter to get started; every other block in the example file enables one more real integration and can stay blank.

Variable	Required?	What it does and where to get it
NVIDIA_API_KEY	Yes	Powers copy generation, pricing and the vision model. Free key from build.nvidia.com - sign up, then 'Get API Key'. Without it the language endpoints return an honest error rather than sample data.
OTP_DEV_ECHO	Yes, in dev	Set to true. With no SMS provider the login code is shown on screen instead of texted. The OTP itself is still real - random, salted-hashed, expiring, rate-limited. Never enable in production.
PUBLIC_BASE_URL	For phone testing	Your laptop's LAN IP, e.g. http://192.168.1.5:8000. This is what storefront listing URLs point at, so a buyer on another device can open them.
Razorpay, ONDC, Amazon, GeM, Shopify, Shiprocket, MSG91	No	Each enables one real integration. Left blank, that channel reports <i>not_configured</i> and names the exact variables it needs - it never fabricates a success.

10.4 Run

```
# backend - bind to 0.0.0.0, not localhost, or the phone cannot reach it
cd backend
./venv/Scripts/python.exe -m uvicorn main:app --host 0.0.0.0 --port 8000

# app
cd app
npx expo start --web
```

Open <http://localhost:8081> for the app, or <http://localhost:8000> for a phone-shaped frame with a live backend status bar. Check `/health` returns `"llm": "nemotron-3-ultra"`.

On Windows, allow the port once so the phone can connect:

```
New-NetFirewallRule -DisplayName "Kalakriti API" -Direction Inbound `
  -LocalPort 8000 -Protocol TCP -Action Allow
```

10.5 Building for a phone

```
cd app
npx expo prebuild --platform android --clean
cd android

JAVA_HOME="/c/Program Files/Java/jdk-22" \
ANDROID_HOME="$HOME/AppData/Local/Android/Sdk" \
EXPO_PUBLIC_API_URL="http://192.168.1.5:8000" \
  ./gradlew assembleRelease -PreactNativeArchitectures=arm64-v8a
adb install -r app/build/outputs/apk/release/app-release.apk
```

Use your own LAN IP

EXPO_PUBLIC_API_URL is baked in at build time. If you forget it, or your IP changes on the venue Wi-Fi, you do not need to rebuild: open **Profile** -> **Server address** in the app, type the new address and press Test. That escape hatch exists because a release APK has no Metro to infer a LAN address from, and a demo should not die for want of a rebuild.

10.6 Verifying end to end

- Open the app, pick a language, choose *Not now - start without an account*.
- **Add a new product** -> Gallery -> any product photo.
- Wait one to three minutes: OCR, detection and background generation run in sequence. Extracted fields appear with confidence badges, all editable.
- Press **Send**. It asks you to sign in; enter any 10-digit number and the OTP appears on screen because OTP_DEV_ECHO is on.
- Fill in name and address. It publishes to your own storefront and returns a real, openable URL with a working Buy button.

10.7 Problems you will actually hit

Symptom	Cause and fix
MalformedURLException: no protocol	The build has no backend URL. Set EXPO_PUBLIC_API_URL, or use Profile -> Server address.

Symptom	Cause and fix
CLEARTEXT communication not permitted	Android 9+ blocks plain HTTP in release builds; debug builds are exempt, which is why it only appears in the APK. Handled by app/plugins/withLocalNetworkAccess.js, which permits cleartext for private LAN ranges only - not for the whole internet. If it returns, re-run expo prebuild.
Phone cannot reach the laptop	Different networks or a firewall. Same Wi-Fi, add the firewall rule. Mobile data will never work - the laptop has no public address.
"llm": "not configured"	NVIDIA_API_KEY missing, or uvicorn started before .env existed. It is read at startup, so restart.
Analysis takes 2-5 minutes	Nemotron is a reasoning model; this is inherent. Pre-warm one product before presenting.
Metro serves a stale bundle	npx expo start --clear. Symptom: your change is not in the app and the bundle size did not move.

10.8 Changing UI text

No user-facing string is written inline. All 330 live in app/src/i18n/catalog.ts in English and Hindi; the other seven Indian languages are generated from those and committed, so the interface needs no network. After editing:

```
cd app && node scripts/dump-en.mjs          # catalogue -> en.json + hi.json
cd ../backend && python translate_ui.py      # -> gu bn mr ta te kn or
```

A missing string falls back to **English, never Hindi**. That is deliberate: an untranslated Gujarati string showing English is a visible gap somebody will fix, whereas showing Hindi looks like the language switch is broken.

11. Two ways to make a listing

Because the connection an artisan actually has is not the one a demo is built on.

11.1 The problem, stated honestly

The analysis pipeline takes two to five minutes. The obvious design holds an HTTP request open for all of it, and that design is fine on office wifi and useless in a weaving village. A five-minute request does not survive a 2G connection: it times out, the upload dies with it, and the artisan pays for the same megabytes again on the retry. She sees a failure and no explanation.

So there are two modes, and she chooses. The app measures the connection and proposes one; it never decides. A measurement is a guess about the next minute, and somebody who prefers to type should not be argued with by a latency reading.

11.2 Auto - let the AI do it

The photograph is uploaded once, to a job queue. The request returns in under a second and **she can close the app**. The server runs the whole pipeline and writes each stage into the job row as it finishes.

Results come back cheapest first, and that ordering is the point:

Stage	What arrives	Size
1	Fields - title, price, category, HSN, the text read off the label	~1 KB
2	Thumbnail	~15 KB
3	Full image, only when she opens the product	~200 KB

An artisan on a bad connection gets a complete, editable listing from the first kilobyte. The pictures arrive when they can. Jobs live in the database, not in memory, so a deploy or a crash does not lose an upload - anything left running when the server restarts is put back in the queue.

11.3 Manual - she fills it in, AI on request

Manual mode is **not** AI switched off. Every model the automatic path uses is still there. The difference is who starts it, and when.

- Each field carries its own AI button. Tapping it returns a suggestion shown beside what she already wrote, with *Use this* and *Keep mine*. Nothing is written until she accepts.
- She can steer it - shorter, simpler words, more detail, or a free-text instruction - and ask again.
- **Carry on with AI from here** fills everything still empty in one go, and leaves every field she has typed byte-identical. What she wrote is the ground truth the model works from, not something to be improved.

The promise is kept on the server, not just in the interface

In manual mode the upload carries `skipModels: true` and the pipeline genuinely runs nothing - no OCR, no vision model, no background generation. The photograph is stored so the AI buttons have something to look at if she asks. Until she asks, nothing has looked at her product. A mode that quietly analysed anyway would be a lie told in a place nobody would check.

11.4 What she is told, and what we cannot do

A card on the Home tab shows every upload the server is working through, naming the actual stage - reading the label, looking at your product - and shows the finished ones she has not opened yet.

What the app does **not** do is wake itself when it is closed. That needs Firebase Cloud Messaging: a Google project, a google-services.json and an FCM server key. None of that is in this repository, so the app does not claim it. The job endpoints such a notification would be driven from already exist.

12. Taking it off the laptop

What is real already, what is single-machine, and the four things that change.

12.1 "There is no database" is not true

State today	
Database	Real. SQLite through SQLAlchemy - accounts, listings, publications, orders, shipments, enquiries, an append-only event log, guarded status transitions. A real relational database. It is a file on a laptop .
Images	Real files, written to backend/media/. Also on the laptop.
Auth	Real. Random OTP, salted-SHA-256 hashed, expiring, rate-limited, server-side sessions that logout genuinely revokes.
Storefront	Real. A public product page with a working Buy button that creates a real order row. Reachable only on the LAN.

Nothing here is mocked. It is **single-machine**. Four changes move it to a real deployment.

12.2 Postgres instead of a file

SQLite on a hosted container is the failure that hurts most, because it is silent: the container filesystem is wiped on every deploy, so every account, listing and order disappears and nobody notices until somebody tries to log in. No code changes - DATABASE_URL is already read and the driver is in requirements.txt. Neon (0.5 GB, scales to zero) or Supabase (500 MB) are free and not trials.

12.3 Object storage instead of the container disk

The same failure applied to photographs. backend/media.py uses any S3-compatible store when MEDIA_S3_* is set and falls back to disk when it is not. Cloudflare R2 gives 10 GB with **no egress charge**, which is what makes image hosting expensive everywhere else.

12.4 A public HTTPS host

A Dockerfile and a Render blueprint are in the repository. Once the backend has a public URL, the same-Wi-Fi requirement disappears entirely - the phone can be on mobile data anywhere.

The one thing that will bite you

onnxruntime, the U²-Net weights and RapidOCR need roughly **700 MB resident**. A 512 MB free instance is OOM-killed on the first photograph, and the symptom is a restart loop rather than an error. Either pay for a 2 GB instance, or turn the heavy local models off and lean on the hosted vision model - a real trade, and it should be chosen deliberately rather than discovered at the demo.

12.5 Real SMS

OTP_DEV_ECHO prints the code on screen and must never be set in production. It is a development shortcut, not a security one - the OTP is real either way. MSG91 is the usual Indian choice because DLT-registered templates are legally required for transactional SMS here. Without a provider, sms.py returns not_configured and **does not deliver**; verification does not weaken, the code still has to match. It simply cannot be received, which is the truth.

12.6 What needs paperwork, not code

Platform	What it needs	Realistic wait
ONDC	Network-participant registration, an Ed25519 keypair published in the registry, a subscriber ID	Weeks
GeM	Registered seller account with GSTIN and PAN	Days to weeks
Amazon	Professional selling account, LWA app, refresh token; per-listing traffic also needs Brand Registry	Days
Razorpay	KYC on a business entity	Days
Shiprocket	Seller account and a pickup address	Hours

Until each exists it reports `not_configured` and names the exact variables it wants. The demo does not depend on any of them: the storefront channel is fully real, publishes immediately and takes real orders.